

ODT Library

RigL Sparse Training

Complete Gap Analysis & Full Implementation

Line-by-line explanation — every source file, every gap, every fix

Algorithm	RigL (Evci et al., NeurIPS 2020) — dynamic sparse training
Target	STM32 Nucleo-F746ZG — 320 KB RAM, 1 MB flash, Cortex-M7 @ 216 MHz
Goal	90% weight sparsity with near-dense accuracy for speaker ID
Chapters	12 + Glossary — each shows CURRENT code → GAP → IMPLEMENTATION

Mohamed — Master's Thesis, 2026

Chapter 1

RigL Algorithm: What Every File Must Contribute

Understanding the full algorithm before touching any source file

What Is RigL?

RigL (Rigging the Lottery, Evci et al. 2020) trains sparse neural networks from scratch. Unlike pruning (which trains dense then compresses), RigL **discovers** the sparse connectivity pattern during training by periodically exchanging connections based on weight magnitude and gradient magnitude.

CONCEPT: Core Insight

Small $|w|$ = weight contributes little to output (good to prune). Large $|\text{grad}|$ = gradient wants to flow through a connection (good to grow). RigL uses both signals to update which connections **EXIST**, not just their values.

The Four Phases of One RigL Cycle

Phase	Operation	When	Which weights
1. PRUNE	Set K active weights with smallest $ w $ to inactive (mask=0)	Every T steps	Active only
2. GROW	Set K inactive weights with largest $ \text{grad} $ to active (mask=1)	Same time as prune	Inactive only
3. STEP	SGD/AdamW updates ALL parameter values (including inactive!)	Every step	All weights
4. RE-MASK	Force inactive positions back to 0.0 (remove drift)	After every step	Inactive only

Critical Ordering Rule

```
// CORRECT ORDER (must follow exactly):
calculateGradsSequential(...);
rigLModelStep(model, N, &cfg);
optimizer.step(model, N, ...);
rigLReMaskModel(model, N);
optimizer.zero(model, N);
```

Step 1: forward + backward
Step 2: prune+grow (reads grad BEFORE zero)
Step 3: update weight VALUES (all weights)
Step 4: zero pruned positions (remove drift)
Step 5: clear gradients

WARNING

Step 2 **MUST** happen before step 5 (zero). The grow decision uses the current gradient. Step 4 **MUST** happen after step 3 (optimizer step). If done before, optimizer sees zeros. If step 4 is skipped entirely, pruned weights drift away from zero.

What Every ODT File Must Contribute

File	Current State	Required Change
Tensor.h	Empty sparsityConfig stub	Add rigLSparsityConfig_t with mask, targetSparsity, turnoverFrac, updateFreq, numCycles, stepCount, cycleCount
MinMax.h/c	findAbsMaxFloat, findAbsMinFloat no	Add findAbsKthSmallest (for prune) and findAbsKthLargestInactive (for grow)
Matmul.h/c	No mask check in inner loop	Add matmulFloat32TensorsWithMask — skip pruned weights in inner loop
Linear.h/c	linearConfig_t has no mask field	Add weightMask field; sparse forward path checks mask before multiply
Conv1d.h/c	No mask in conv kernel	Add weightMask to conv1dConfig_t; skip [oc,ic,k] if mask bit=0
Sgd.c	Updates ALL params unconditionally	Re-mask after update: if mask[i]==0, force param[i]=0.f
TrainingEpoch.c	No RigL step in training loop	Add rigLModelStep call + rigLReMaskModel after optimizer.step
Serialize.c	serializeSparsity() is empty stub	Save BOOL mask tensors per sparse layer in ODTS format
RigL.h/c	File does not exist	Create src/rigl/RigL.h and RigL.c with all RigL functions

Chapter 2

Tensor.h / Tensor.c — Adding rigLSparsityConfig_t

The sparsity stub that needs to become a real struct

Current Code — What Actually Exists

```
// ---- CURRENT Tensor.h (actual ODT code) ----
```

```
typedef enum {
```

```
    SPARSITY_TYPE_1,
```

```
    SPARSITY_TYPE_2,
```

```
} sparsityType_t;
```

```
typedef struct sparsityConfig {
```

```
    /* EMPTY */
```

```
} sparsityConfig_t;
```

```
typedef struct {
```

```
    sparsityType_t type;
```

```
    sparsityConfig_t *config;
```

```
} sparsity_t;
```

```
typedef struct tensor {
```

```
    uint8_t      *data;
```

```
    shape_t      *shape;
```

```
    quantization_t *quantization;
```

```
    sparsity_t    *sparsity;
```

```
} tensor_t;
```

Placeholder — unused, no implementation

Placeholder — unused, no implementation

No fields at all — a stub

Pointer to empty struct — useless

Field exists but nothing populates it

WARNING

The tensor_t struct already has a sparsity field — the plumbing is in place! But the config struct is empty and no code writes to it. This is a well-planned gap.

The Gap

```
// THREE SPECIFIC GAPS:
```

```
// Gap 1: sparsityType_t has placeholder names
```

```
// Gap 2: sparsityConfig_t is empty
```

```
// Gap 3: no init function, no applyMask()
```

Should be RIGL and maybe UNSTRUCTURED

Needs rigLSparsityConfig_t fields

No API to use the sparsity field

Fix 1 — Replace sparsityType_t

```
typedef enum {
```

```
    SPARSITY_RIGL,
```

```
    SPARSITY_UNSTRUCTURED,
```

```
} sparsityType_t;
```

RigL dynamic sparse training (Evci 2020)

Static unstructured pruning (future)

Fix 2 — Define rigLSparsityConfig_t

```
typedef struct rigLSparsityConfig {
```

```
    tensor_t *mask;
```

```
    float    targetSparsity;
```

```
    float    turnoverFrac;
```

```
    size_t   updateFreq;
```

```
    size_t   numCycles;
```

```
    size_t   stepCount;
```

```
    size_t   cycleCount;
```

```
} rigLSparsityConfig_t;
```

BOOL tensor, same shape as weights — 1=active, 0=pruned

e.g. 0.90 = 90% of weights are inactive

e.g. 0.20 = 20% of active weights swapped per cycle

T: steps between prune+grow updates (e.g. 100)

C: total prune+grow cycles before mask is frozen

Internal: steps since last mask update

Internal: completed cycles

CONCEPT: Why stepCount AND cycleCount?

stepCount counts gradient steps since the last mask update (reset to 0 after each update). cycleCount counts how many prune+grow cycles have completed. When cycleCount >= numCycles, the mask is frozen — no more updates. This gives RigL a warmup phase (training with a fixed sparse structure at the end).

BOOL Mask: How 1 Bit Per Weight Works

```
// BOOL tensor packs 8 bits per byte:
```

```
// mask->data = uint8_t array of ceil(N/8) bytes
```

```
// Bit i is at: byte[i/8], bit position (i % 8)
```

```
// Read mask bit i:
```

```
bool tensorBoolGet(tensor_t *t, size_t i) {
```

```
    return (t->data[i/8] >> (i%8)) & 1;
```

```
}
```

```
// Write mask bit i:
```

```
void tensorBoolSet(tensor_t *t, size_t i, bool v) {
```

```
    if (v) t->data[i/8] |= (1u << (i%8));
```

```
    else  t->data[i/8] &= ~(1u << (i%8));
```

```
}
```

```
// Memory savings example:
```

```
// Linear(256, 128): N = 32768 weights
```

```
// BOOL mask = 32768/8 = 4096 bytes = 4 KB
```

Shift and mask to get bit

Set bit

Clear bit

vs 128 KB for float32 weights

TIP

For a Linear(256,128) layer: 32768 weights × 4 bytes = 128 KB float storage. The BOOL mask adds only 4 KB (3% overhead). The 90% sparsity saves 90%×128 KB = 115 KB of EFFECTIVE computation — you skip 90% of multiply-adds in the forward pass.

Chapter 3

MinMax.h / MinMax.c — K-th Smallest & Largest for Prune/Grow

Current code has findAbsMax but not findAbsKthSmallest — the core selection algorithm

Current MinMax.h — What Exists

<pre>// --- ACTUAL CURRENT MinMax.h --- float absFloat32(float a); float maxFloat32s(float a, float b); float findAbsMaxFloat(uint8_t *bytes, size_t n); float findMaxFloat(uint8_t *bytes, size_t n); float findMinFloat(uint8_t *bytes, size_t n); int32_t findMaxInt32(uint8_t *bytes, size_t n); int32_t findMinInt32(uint8_t *bytes, size_t n); // MISSING: // findAbsMinFloat() — doesn't exist! // findAbsKthSmallest() — doesn't exist! // findAbsKthLargestInactive() — doesn't exist!</pre>	<i>Wrapper for fabsf()</i> <i>Max of two floats</i> <i>Find maximum x over all N elements</i> <i>Max (signed)</i> <i>Min (signed)</i> <i>Needed for pruning</i> <i>Needed for K-th smallest threshold</i> <i>Needed for K-th largest grad among inactive</i>
--	--

Current findAbsMaxFloat — The Reference Implementation Pattern

This is the actual code. Study it to understand the pattern for the new functions:

<pre>float findAbsMaxFloat(uint8_t *bytes, size_t n) { float *values = (float *)bytes; float max = fabsf(values[0]); for (size_t i = 1; i < n; i++) { if (fabsf(values[i]) > max) { max = fabsf(values[i]); } } return max; }</pre>	<i>Cast uint8_t* to float* (big trust)</i> <i>Initialize to first element's x</i> <i>Scan from index 1 onwards</i> <i>Track running maximum</i> <i>$O(N)$ time, $O(1)$ space</i>
---	--

WARNING

Note: findAbsMaxFloat takes uint8_t* bytes, not float*. This is because ODT tensors store data as raw bytes. The cast (float *)bytes is safe because the underlying storage IS float32.

New Function 1: findAbsMinFloat — Needed for Prune

We need the minimum absolute value to understand the range for pruning selection:

<pre>float findAbsMinFloat(float *arr, size_t n) { float mn = fabsf(arr[0]); for (size_t i = 1; i < n; i++) { float a = fabsf(arr[i]); if (a < mn) mn = a; } return mn; }</pre>	<i>Initialize to first x </i> <i>Track running minimum</i> <i>O(N) time, O(1) space</i>
---	---

New Function 2: findAbsKthSmallestActive — The Prune Threshold

This is the most important new function. It finds the K-th smallest $|w|$ among ACTIVE weights (mask=1). This threshold is used to prune the K smallest active weights:

<pre>float findAbsKthSmallestActive(float *arr, tensor_t *mask, size_t n, size_t k) { float lastFound = -1.f; float kth = 0.f; for (size_t ki = 0; ki < k; ki++) { float cur = 1e38f; for (size_t i = 0; i < n; i++) { if (!tensorBoolGet(mask, i)) continue; float a = fabsf(arr[i]); if (a > lastFound && a < cur) cur = a; } lastFound = cur; kth = cur; } return kth; }</pre>	<i>k = number of weights to prune</i> <i>Largest value found in previous passes</i> <i>k passes: find 1st, 2nd, ..., k-th smallest</i> <i>Infinity sentinel</i> <i>Skip inactive weights</i> <i>Strictly greater than last found, less than cur</i> <i>Update candidate</i> <i>This pass found the (ki+1)-th smallest</i> <i>The K-th smallest w among active weights</i>
---	---

CONCEPT: Why selection-sort instead of quickselect?

QuickSelect is $O(N)$ average but requires $O(N)$ stack space for the partition buffer. On a 320 KB MCU with 128 KB already used for model weights, there may not be N floats available. Selection-sort is $O(k*N)$ time but $O(1)$ space — no allocation needed. For typical RigL: $k = \text{turnoverFrac} * \text{numActive} = 0.20 * 0.10 * N = 0.02*N$. Total work = $0.02*N * N = 0.02*N^2$. For $N=32768$: ~21M ops. At 216 MHz: ~10ms per cycle. With updateFreq=100 steps, this runs once per 100 batches — negligible overhead.

New Function 3: findAbsKthLargestInactive — The Grow Threshold

<pre>float findAbsKthLargestInactive(float *grads, tensor_t *mask, size_t n, size_t k) { float lastFound = 1e38f;</pre>	<i>k = number of weights to grow</i> <i>Largest value found in previous passes</i>
---	--

```

float kth = 0.f;
for (size_t ki = 0; ki < k; ki++) {
    float cur = -1.f;
    for (size_t i = 0; i < n; i++) {
        if (tensorBoolGet(mask, i))
            continue;

        float a = fabsf(grads[i]);
        if (a < lastFound && a > cur)
            cur = a;
    }
    lastFound = cur;
    kth = cur;
}
return kth;
}

```

k passes: find 1st, 2nd, ..., k-th largest

Negative infinity sentinel

Skip ACTIVE weights

We want inactive only

Strictly less than last found, more than cur

Update candidate

This pass found the (ki+1)-th largest

The K-th largest |grad| among inactive

TIP

PRUNE uses smallest $|w|$ among ACTIVE (mask=1). GROW uses largest $|grad|$ among INACTIVE (mask=0). These are symmetric selection problems, solved by the same iterative scan pattern.

Chapter 4

Matmul.h / Matmul.c — Mask-Aware Matrix Multiply

The current inner loop reads ALL weights — we need to skip pruned ones

Current matmulFloatCore — The Actual Inner Loop

Here is the ACTUAL current inner loop from Matmul.c:

<pre>// --- Current matmulFloatCore (simplified) --- for (r=0; r<aRows; r++) { for (c=0; c<bColumns; c++) { float result = bias ? bias[c] : 0.f; for (size_t i = 0; i < aColumns; i++) { size_t aIdx[] = {r, i}; float aVal = readFloat(aTensor, aIdx); size_t bIdx[] = {i, c}; float bVal = readFloat(bTensor, bIdx); result += mulFloat32s(aVal, bVal); } output[r*bColumns + c] = result; } }</pre>	<i>Initialize accumulator with bias</i> <i>Read input element</i> <i>Read weight element (ALL weights!)</i> <i>Multiply and accumulate — no mask check!</i>
---	---

WARNING

The key line is 'result += mulFloat32s(aVal, bVal)' — it runs for EVERY weight, including ones that should be pruned (mask=0). At 90% sparsity, 90% of these multiplications are wasted (multiplying by ~0.0). RigL gives no speedup without this fix.

New Function: matmulFloat32TensorsWithMask

<pre>void matmulFloat32TensorsWithMask(tensor_t *a, tensor_t *b, tensor_t *out, tensor_t *bias, tensor_t *mask) { size_t aRows = getDimensionsByIndex(a, 0); size_t aCol = getDimensionsByIndex(a, 1); size_t bCol = getDimensionsByIndex(b, 1); float *bData = (float *)b->data; float *bsA = bias ? (float *)bias->data : NULL; for (size_t r = 0; r < aRows; r++) { for (size_t c = 0; c < bCol; c++) { float acc = bsA ? bsA[c] : 0.f; for (size_t k = 0; k < aCol; k++) { size_t bFlat = k*bCol + c; if (mask && !tensorBoolGet(mask, bFlat))</pre>	<i>BOOL mask tensor, or NULL for dense</i> <i>= bRows = inFeatures</i> <i>= outFeatures</i> <i>Weight matrix row pointer</i> <i>Each output row (batch item)</i> <i>Each output column (out feature)</i> <i>Init with bias[c]</i> <i>Each input feature</i> <i>Flat weight index [k, c]</i> <i>Is this weight pruned?</i>
--	---

<pre> continue; float aV = ((float*)a->data)[r*aCol+k]; acc += aV * bData[bFlat]; } ((float*)out->data)[r*bCol+c] = acc; } } } </pre>	<i>YES — skip multiply-add entirely</i> <i>Read input</i> <i>Multiply-accumulate (only active)</i> <i>Write output</i>
---	--

CONCEPT: Why is this a speedup?

At 90% sparsity, 90% of the inner-loop iterations hit 'continue' immediately — just one tensorBoolGet call (bit extraction) instead of a full multiply-add. FLOPs saved: $0.9 * aRows * aCol * bCol$ multiply-adds. For Linear(256,128): $0.9 * 1 * 256 * 128 = 29,491$ FLOPs saved per forward pass. On Cortex-M7 at 216 MHz with single-cycle FMA: ~0.14 ms saved per batch.

Chapter 5

Linear.h / Linear.c / LinearApi — Adding weightMask

The sparse linear layer: adding the mask field and a new sparse init function

Current linearConfig_t — What's Missing

```
// --- Current Linear.h (actual code) ---
```

```
typedef struct linearConfig {  
    parameter_t *weights;  
    parameter_t *bias;  
    arithmetic_t forwardMath;  
    arithmetic_t weightGradMath;  
    arithmetic_t biasGradMath;  
    arithmetic_t propLossMath;  
    quantization_t *outputQ;  
    quantization_t *propLossQ;  
    outputMode_t weightGradAccMode;  
    outputMode_t biasGradAccMode;  
    bool ownsQuantizations;  
    // MISSING: tensor_t *weightMask;  
} linearConfig_t;
```

Weight parameter (param + grad tensors)

Bias parameter (or NULL if no bias)

No mask field — RigL cannot work without this

Fix: Add weightMask to linearConfig_t

```
typedef struct linearConfig {  
    parameter_t *weights;  
    parameter_t *bias;  
    arithmetic_t forwardMath;  
    arithmetic_t weightGradMath;  
    arithmetic_t biasGradMath;  
    arithmetic_t propLossMath;  
    quantization_t *outputQ;  
    quantization_t *propLossQ;  
    outputMode_t weightGradAccMode;  
    outputMode_t biasGradAccMode;  
    bool ownsQuantizations;  
    tensor_t *weightMask;  
    rigLSparsityConfig_t *sparsityCfg;  
} linearConfig_t;
```

NEW: BOOL tensor or NULL (dense layer)

NEW: RigL schedule config or NULL

Fix: Sparse Forward Path in linearForwardFloat

```
static void linearForwardFloat(linearConfig_t *c,  
    tensor_t *input, tensor_t *output) {  
    tensor_t *W = getParamFromParameter(c->weights);
```

```

tensor_t *b = c->bias ?
    getParamFromParameter(c->bias) : NULL;

if (c->weightMask != NULL) {
    matmulFloat32TensorsWithMask(
        input, w, output, b,
        c->weightMask);
} else {
    matmulFloat32TensorsWithBias(
        input, w, output, b);
}
}

```

Sparse path — mask present

Pass mask — skip pruned weights

Dense path — unchanged

No mask — all weights used

CONCEPT: NULL = dense, non-NULL = sparse

Setting weightMask=NULL gives the original dense behavior with zero overhead. This is critical for layers you DON'T want to sparsify (e.g., the final classifier). You can mix sparse and dense layers in the same model.

New API: linearLayerInitSparse

```

void linearLayerInitSparse(linearConfig_t *c,
    float targetSparsity,
    tensor_t *maskBuffer,
    uint8_t *maskData,
    size_t updateFreq,
    size_t numCycles,
    riglSparsityConfig_t *sparsityCfg) {
    // 1. Init mask tensor (all active initially)
    size_t n = calcNumberOfElementsByParameter(c->weights);
    setTensorValues(maskBuffer, BOOL, maskData, n, 1);
    c->weightMask = maskBuffer;
    // 2. Prune to target sparsity immediately
    riglInitMaskFromWeights(c->weights->param,
        c->weightMask, targetSparsity);
    // 3. Wire sparsity config
    sparsityCfg->mask = c->weightMask;
    sparsityCfg->targetSparsity = targetSparsity;
    sparsityCfg->updateFreq = updateFreq;
    sparsityCfg->numCycles = numCycles;
    sparsityCfg->stepCount = 0;
    sparsityCfg->cycleCount = 0;
    c->sparsityCfg = sparsityCfg;
}

```

e.g. 0.90

Caller-allocated BOOL tensor

Caller-allocated mask byte array

T: steps between prune+grow

C: total mask update cycles

Caller-allocated config struct

All 1s = fully dense start

Wire mask to config

Prune K smallest |w| to reach 90%

TIP

All buffers (maskBuffer, maskData, sparsityCfg) are caller-allocated. No malloc inside — safe for MCU. Declare them as static arrays in your application.

Chapter 6

Conv1d.h/c and Conv1dTransposed — Sparse Convolution

Adding mask support to the convolutional inner loop

The Conv1d Gap

```
// Current conv1dKernelFloat32 inner loop (simplified):
for (oc=0; oc<outCh; oc++)
  for (t=0; t<T_out; t++)
    float acc = bias ? bias[oc] : 0.f;
    for (ic=0; ic<inChPerGroup; ic++)
      for (k=0; k<kSize; k++) {
        size_t wFlat = oc*inChPerGroup*kSize + ic*kSize + k;
        // NO MASK CHECK HERE:
        acc += input[...] * W[wFlat];
      }
```

Flat weight index
Gap: all weights used

Fix: Mask-Aware Conv1d Inner Loop

```
for (oc=0; oc<outCh; oc++)
  for (t=0; t<T_out; t++)
    float acc = bias ? bias[oc] : 0.f;
    for (ic=0; ic<inChPerGroup; ic++)
      for (k=0; k<kSize; k++) {
        size_t wFlat = oc*inChPerGroup*kSize + ic*kSize+k;
        if (c->weightMask &&
            !tensorBoolGet(c->weightMask, wFlat))
          continue;
        acc += input[b*T_in*inCh + (t+k)*inCh + ic]
              * W[wFlat];
      }
```

For each output channel
For each output time step

For each input channel in group
For each kernel position
Flat weight index

Is weight [oc,ic,k] pruned?
YES — skip entirely

Multiply-accumulate (active only)

CONCEPT: Conv1d mask shape vs Linear mask shape

Linear mask: [outFeatures, inFeatures] — one bit per weight matrix entry. Conv1d mask: [outChannels, inChPerGroup, kernelSize] — one bit per filter weight. For Conv1d(in=32, out=64, kernel=3, groups=1): $N = 64 \times 32 \times 3 = 6144$ weights, mask = 768 bytes. For grouped conv (groups=4): inChPerGroup = 8, $N = 64 \times 8 \times 3 = 1536$ weights, mask = 192 bytes.

Chapter 7

Optimizer.h / Sgd.h / Sgd.c — Re-Masking After Step

The optimizer updates ALL weights — we must zero pruned positions afterward

Current sgdUpdateKernel — The Actual Code

```
// --- Actual Sgd.c sgdUpdateKernel ---
static void sgdUpdateKernel(tensor_t **op,
    size_t n, tensor_t *rawOut,
    tensor_t *aux, const void *ctxv) {
    const sgdUpdateCtx_t *ctx = ctxv;
    size_t nVals = calcNumberOfElementsByTensor(rawOut);
    float *param = (float *)op[0]->data;
    float *grad = (float *)op[1]->data;
    float *out = (float *)rawOut->data;
    for (size_t i = 0; i < nVals; i++) {
        float g = grad[i] + ctx->weightDecay * param[i];
        out[i] = param[i] - ctx->lr * g;
    }
}
```

ctx has lr and weightDecay

Current weight values

Current gradient values

Output buffer (same as param for in-place)

ALL elements — no mask check!

Gradient with weight decay

*SGD step: $w \leftarrow lr * (g + wd * w)$*

Pruned weights now have nonzero values!

WARNING

After this loop, a weight that should be zero (mask=0) now has: $\text{new_val} = 0.0 - lr * (\text{grad} + wd * 0.0) = -lr * \text{grad}$. The gradient is non-zero because backprop still computes gradients for ALL positions. Without re-masking, these 'ghost' weights grow over time and corrupt the model.

Why Allow Inactive Weights to Accumulate Gradient?

CONCEPT: The RigL design choice: compute ALL gradients

Option A: Zero gradients for inactive weights (in backward pass). Problem: The GROW step needs to compare $|\text{grad}|$ across inactive positions. If we zero their gradients, we lose the signal used to decide which weights to grow. Option B: Compute ALL gradients (current approach). Then re-mask after optimizer step. This way: (1) grow step has real $|\text{grad}|$ for all inactive positions, (2) re-mask prevents inactive weights from contributing to the NEXT forward pass.

Fix: rigLReMaskTensor — Applied After Every Optimizer Step

```
void rigLReMaskTensor(tensor_t *weights, tensor_t *mask) {
    if (mask == NULL) return;
    size_t n = calcNumberOfElementsByTensor(weights);
    float *wArr = (float *)weights->data;
    for (size_t i = 0; i < n; i++) {
        if (!tensorBoolGet(mask, i)) {
            wArr[i] = 0.f;
        }
    }
}
```

Dense layer — nothing to do

Is this weight inactive?

YES — force back to zero

```

    }
}
// Called for ALL sparse layers after optimizer.step():
void rigLReMaskModel(layer_t **model, size_t N) {
    for (size_t i = 0; i < N; i++) {
        if (model[i]->type == LINEAR) {
            linearConfig_t *c = model[i]->config->linear;
            if (c->weightMask)
                rigLReMaskTensor(
                    getParamFromParameter(c->weights),
                    c->weightMask);
        }
        // ... similar for CONV1D layers
    }
}

```

Only if this layer is sparse

TIP

rigLReMaskTensor is $O(N)$ — one pass through the weight array. For a 32768-weight linear layer: 32768 comparisons. At 216 MHz: ~0.15 ms. This runs after EVERY optimizer step (not just mask update steps).

Chapter 8

TrainingEpochDefault.c / CalculateGradsSequential.c — RigL Integration

Wiring RigL into the training loop: where to call rigLModelStep and rigLReMaskModel

Current TrainingEpochDefault — No RigL

```
// Simplified current trainingEpochDefault:
for (size_t bi=0; bi<numBatches; bi++) {
    batch_t *batch = getBatch(loader, bi);
    calculateGradsSequential(model, N,
        loss, REDUCTION_MEAN, batch, ...);
    optimFns.step(optimizer);
    optimFns.zero(optimizer);
    freeBatch(batch);
}
```

Forward + backward

Update weights — NO RigL step!

Clear grads — NO re-mask!

Fixed: TrainingEpochDefault with RigL

```
for (size_t bi=0; bi<numBatches; bi++) {
    batch_t *batch = getBatch(loader, bi);

    /* Step 1: compute gradients */
    calculateGradsSequential(model, N,
        loss, REDUCTION_MEAN, batch, ...);

    /* Step 2: RigL prune+grow (before zero)*/
    if (rigLCfg != NULL) {
        rigLModelStep(model, N, rigLCfg);
    }

    /* Step 3: optimizer weight update */
    optimFns.step(optimizer);

    /* Step 4: re-mask (after step!) */
    if (rigLCfg != NULL) {
        rigLReMaskModel(model, N);
    }

    /* Step 5: zero gradients */
    optimFns.zero(optimizer);
    freeBatch(batch);
}
```

Grads computed for ALL weights

Only if RigL is enabled

Check schedule; if time: prune+grow

SGD/AdamW updates ALL weights

Zero inactive positions after drift

Clear ALL gradients

rigLModelStep — The Scheduling Logic

```
void rigLModelStep(layer_t **model,
    size_t N, rigLModelConfig_t *cfg) {
```



```

cfg->globalStep++;

/* Mask frozen after numCycles? */
if (cfg->cyclesDone >= cfg->numCycles)
    return;

/* Is it time for a mask update? */
if (cfg->globalStep % cfg->updateFreq != 0)
    return;

/* Compute K = floor(turnoverFrac * nnz)*/
size_t nnz = rigLCountActive(cfg);
size_t K = (size_t)(cfg->turnoverFrac * (float)nnz);
if (K == 0) K = 1;

/* Per-layer prune+grow */
for (size_t i = 0; i < N; i++) {
    if (model[i]->type == LINEAR) {
        linearConfig_t *c = model[i]->config->linear;
        if (c->weightMask)
            rigLLinearStep(c, K);
    }
    // ... CONV1D similar
}
cfg->cyclesDone++;
}

```

Count every gradient step

Mask frozen — no more updates

Not yet — wait for next multiple of T

Count active (mask=1) weights

Always swap at least 1

Only sparse layers

Prune K + grow K

Increment completed cycles

CONCEPT: Worked example: updateFreq=100, numCycles=10, N=32768

Steps 1-99: no mask update. Step 100: first mask update. $nnz = 0.10 * 32768 = 3277$. $K = 0.20 * 3277 = 655$. Prune 655 smallest $|w|$ active weights. Grow 655 largest $|grad|$ inactive weights. cyclesDone=1.
Steps 101-199: no mask update. Step 200: second update. cyclesDone=2. ... Step 1000: 10th update. cyclesDone=10. MASK FROZEN. Steps 1001+: train with fixed mask (no more prune/grow).

Chapter 9

Layer.h / Serialize.h/.c — Layer Sparsity Query & Mask Serialization

Querying whether a layer is sparse and saving masks to the ODTS binary format

Layer.h — getLayerMask Helper

```
/* Returns the BOOL mask tensor for a layer, or NULL if dense */
/
tensor_t *getLayerMask(layer_t *layer) {
    switch (layer->type) {
        case LINEAR:
            return layer->config->linear->weightMask;
        case CONV1D:
            return layer->config->conv1d->weightMask;
        case CONV1D_TRANSPOSED:
            return layer->config->conv1dTransposed->weightMask;
        default:
            return NULL;
    }
}
```

Pooling, Norm, ReLU etc. are never sparse

Serialize.c — The Empty serializeSparsity Stub

```
// CURRENT Serialize.c (actual code):
// TODO
static void serializeSparsity() {}
```

Empty — masks are NOT saved!

WARNING

This empty stub means: if you train a RigL model, save it, then reload it, the mask is LOST. On reload, Deserialize will initialize all mask bits to 1 (fully dense). The weights are loaded correctly, but the model now thinks all weights are active — the sparsity structure is completely gone.

Fix: serializeMasks and deserializeMasks

```
/* Format: for each layer i in [0, N):
- If dense (no mask): write nothing
- If sparse:
    uint8_t layer_idx (1 byte)
    uint32_t mask_n_bytes (4 bytes)
    uint8_t mask_data[mask_n_bytes]
End: sentinel layer_idx = 0xFF */
void serializeMasks(layer_t **model, size_t N, FILE *f) {
    for (size_t i = 0; i < N; i++) {
        tensor_t *mask = getLayerMask(model[i]);
```

Which layer

ceil(N/8) bytes for N weights

The packed BOOL mask

<pre>if (!mask) continue; size_t n = calcNumberOfElementsByTensor(mask); size_t maskBytes = (n + 7) / 8; serialWriteU8((uint8_t)i, f); serialWriteU32LE(maskBytes, f); serialWriteBytes(mask->data, maskBytes, f); } serialWriteU8(0xFF, f); }</pre>	<i>Dense layer — skip</i> <i>Packed BOOL: $\text{ceil}(N/8)$ bytes</i> <i>Layer index</i> <i>Byte count</i> <i>Mask bits</i> <i>Sentinel: end of mask section</i>
---	--

TIP

For a 2-layer model (Linear(256,128) + Linear(128,10)) with only layer 0 sparse: layer 0: 1 + 4 + 4096 = 4101 bytes. Sentinel: 1 byte. Total mask overhead: 4102 bytes.

Chapter 10

RigL.h — Complete New Header (Does Not Exist Yet)

Create src/rigl/include/Rigl.h — all types and function declarations

File Header and Includes

<pre>#ifndef RIGL_H #define RIGL_H #include <stddef.h> #include <stdint.h> #include "Layer.h" #include "Tensor.h" #include "Linear.h" #include "Conv1d.h" /* All RigL functions use ARITH_FLOAT32 arithmetic. */ /* Integer-arithmetic arms are future work (#326). */</pre>	<i>Include guard</i> <i>size_t</i> <i>uint8_t, uint32_t</i> <i>layer_t, layerType_t</i> <i>tensor_t, tensorBoolGet/Set</i> <i>linearConfig_t</i> <i>conv1dConfig_t</i>
---	---

Type Definitions

<pre>typedef struct rigLSparsityConfig { tensor_t *mask; float targetSparsity; float turnoverFrac; size_t updateFreq; size_t numCycles; size_t stepCount; size_t cycleCount; } rigLSparsityConfig_t; typedef struct rigLModelConfig { size_t updateFreq; size_t numCycles; float turnoverFrac; size_t globalStep; size_t cyclesDone; } rigLModelConfig_t;</pre>	<i>Per-layer config</i> <i>BOOL tensor, same shape as weights</i> <i>e.g. 0.90</i> <i>e.g. 0.20</i> <i>T</i> <i>C</i> <i>Internal counter</i> <i>Internal counter</i> <i>Model-level schedule</i> <i>T: steps between prune+grow</i> <i>C: total cycles</i> <i>Delta: fraction of active weights swapped</i> <i>Internal: total steps since init</i> <i>Internal: completed cycles</i>
---	--

Function Declarations

<pre>/* ---- Mask Initialization ---- */ void rigLInitSparsityConfig(rigLSparsityConfig_t *cfg, tensor_t *mask, float targetSparsity, float turnoverFrac,</pre>	
--	--

```

    size_t updateFreq,
    size_t numCycles);
void rigLInitMaskFromWeights(
    tensor_t *weights,
    tensor_t *mask,
    float targetSparsity);

/* ---- Core Prune/Grow ---- */
void rigLUpdateMask(
    tensor_t *weights,
    tensor_t *grads,
    tensor_t *mask,
    size_t numToSwap);

/* ---- Selection Helpers ---- */
float rigLFindKthSmallestAbsActive(
    float *arr, tensor_t *mask,
    size_t n, size_t k);
float rigLFindKthLargestAbsInactive(
    float *arr, tensor_t *mask,
    size_t n, size_t k);

/* ---- Re-mask ---- */
void rigLApplyMask(tensor_t *weights, tensor_t *mask);
void rigLReMaskModel(layer_t **model, size_t N);

/* ---- Model Step ---- */
void rigLModelStep(layer_t **model,
    size_t N, rigLModelConfig_t *cfg);
void rigLLinearStep(linearConfig_t *c, size_t K);
void rigLConv1dStep(conv1dConfig_t *c, size_t K);

/* ---- Counting ---- */
size_t rigLCountActive(tensor_t *mask);
#endif // RIGL_H

```

Prune to target sparsity immediately

K: weights to prune AND grow

Force inactive to 0

Call after every optimizer.step()

Prune+grow all sparse layers if scheduled

Count mask=1 bits

Chapter 11

RigL.c — Complete New Implementation (Does Not Exist Yet)

Create src/rigl/RigL.c — every function implemented with line-by-line explanation

11.1 — File Header and Includes

<pre>#define SOURCE_FILE "RIGL" #include <math.h> #include <stddef.h> #include <stdlib.h> #include "Common.h" #include "Layer.h" #include "Linear.h" #include "Conv1d.h" #include "RigL.h" #include "Tensor.h"</pre>	<i>For PRINT_ERROR macro</i> <i>fabsf</i> <i>size_t</i> <i>exit</i> <i>PRINT_ERROR</i> <i>layer_t</i> <i>linearConfig_t</i> <i>conv1dConfig_t</i> <i>tensorBoolGet/Set</i>
--	---

11.2 — rigLInitSparsityConfig

<pre>void rigLInitSparsityConfig(rigLSparsityConfig_t *cfg, tensor_t *mask, float targetSparsity, float turnoverFrac, size_t updateFreq, size_t numCycles) { if (mask == NULL) { PRINT_ERROR("mask must not be NULL"); exit(1); } if (targetSparsity < 0.f targetSparsity >= 1.f) { PRINT_ERROR("targetSparsity must be in [0,1)"); exit(1); } if (updateFreq == 0) { PRINT_ERROR("updateFreq must be > 0"); exit(1); } cfg->mask = mask; cfg->targetSparsity = targetSparsity; cfg->turnoverFrac = turnoverFrac; cfg->updateFreq = updateFreq; cfg->numCycles = numCycles; cfg->stepCount = 0;</pre>	<i>1.0 would mean ALL weights pruned = nothing</i> <i>0 would trigger update every step</i> <i>Reset counters</i>
--	--

```

    cfg->cycleCount = 0;
}

```

11.3 — rigLInitMaskFromWeights: Initial Sparsification

<pre> void rigLInitMaskFromWeights(tensor_t *weights, tensor_t *mask, float targetSparsity) { size_t n = calcNumberOfElementsByTensor(weights); float *wArr = (float *)weights->data; /* Set mask all-1 initially (all active) */ for (size_t i=0; i<n; i++) tensorBoolSet(mask, i, true); /* Prune floor(targetSparsity * n) smallest w */ size_t numToPrune = (size_t)(targetSparsity * n); float threshold = rigLFindKthSmallestAbsActive(wArr, mask, n, numToPrune); size_t pruned = 0; for (size_t i=0; i<n; i++) { if (pruned < numToPrune && fabsf(wArr[i]) <= threshold) { tensorBoolSet(mask, i, false); wArr[i] = 0.f; pruned++; } } } </pre>	<i>e.g. 0.90</i> <i>Total weights</i> <i>Start fully dense</i> <i>e.g. 0.90*32768 = 29491</i> <i>Find the threshold</i> <i>Deactivate</i> <i>Zero the weight too</i>
--	---

CONCEPT: Why initialize to all-1 then prune?

An alternative is to initialize the mask to the target sparsity directly (random pattern). RigL's original paper uses magnitude-based pruning at initialization: keep the largest $|w|$ weights active. This tends to work slightly better than random initialization because high-magnitude weights have already been selected by the normal distribution of weight initializers (Xavier/Glorot) to be 'important'.

11.4 — rigLFindKthSmallestAbsActive: $O(k*N)$ Selection

<pre> float rigLFindKthSmallestAbsActive(float *arr, tensor_t *mask, size_t n, size_t k) { float lastFound = -1.f; float kth = 0.f; for (size_t ki = 0; ki < k; ki++) { float cur = 1e38f; for (size_t i = 0; i < n; i++) { if (!tensorBoolGet(mask, i)) continue; </pre>	<i>Largest abs value found in previous passes</i> <i>k outer iterations</i> <i>Start each pass at +infinity</i> <i>N inner iterations</i> <i>Skip inactive</i>
--	--

<pre> float a = fabsf(arr[i]); if (a > lastFound && a < cur) cur = a; } if (cur >= 1e37f) break; lastFound = cur; kth = cur; } return kth; } </pre>	<i>Strictly > lastFound (avoid re-picking)</i> <i>New candidate for this pass</i> <i>Fewer than k active weights — done early</i> <i>Found ki-th smallest</i> <i>The k-th smallest abs value among active weights</i>
--	---

CONCEPT: Complexity analysis for the thesis

Time: $O(k * N)$ where $k = \text{turnoverFrac} * \text{numActive} = \text{turnoverFrac} * (1 - \text{sparsity}) * N$. At 90% sparsity, 20% turnover: $k = 0.20 * 0.10 * N = 0.02 * N$. Total = $0.02 * N * N = 0.02 * N^2$. For $N=32768$ (Linear 256→128): $0.02 * 32768^2 = 21.5\text{M}$ comparisons per cycle. Each comparison = 1 tensorBoolGet + 1 fabsf + 2 float compares ≈ 5 cycles. Total $\approx 107\text{M}$ cycles. At 216 MHz: ≈ 0.5 seconds per prune+grow cycle. With updateFreq=100, this runs every 100 batches. If batch=0.1s: overhead = 5%. Acceptable.

11.5 — rigLUpdateMask: The Combined Prune+Grow Step

<pre> void rigLUpdateMask(tensor_t *weights, tensor_t *grads, tensor_t *mask, size_t numToSwap) { size_t n = calcNumberOfElementsByTensor(weights); float *wArr = (float *)weights->data; float *gArr = (float *)grads->data; if (numToSwap == 0) return; /* === PRUNE: deactivate K smallest w active === */ float pruneThresh = rigLFindKthSmallestAbsActive(wArr, mask, n, numToSwap); size_t pruned = 0; for (size_t i=0; i<n && pruned<numToSwap; i++) { if (!tensorBoolGet(mask, i)) continue; if (fabsf(wArr[i]) <= pruneThresh) { tensorBoolSet(mask, i, false); wArr[i] = 0.f; pruned++; } } /* === GROW: activate K largest grad inactive === */ float growThresh = rigLFindKthLargestAbsInactive(gArr, mask, n, numToSwap); size_t grown = 0; for (size_t i=0; i<n && grown<numToSwap; i++) { if (tensorBoolGet(mask, i)) continue; if (fabsf(gArr[i]) >= growThresh) { tensorBoolSet(mask, i, true); </pre>	<i>K: prune K AND grow K</i> <i>Nothing to do</i> <i>Find the prune threshold</i> <i>Skip already inactive</i> <i>At or below threshold?</i> <i>Deactivate</i> <i>Zero immediately</i> <i>Find the grow threshold</i> <i>Skip already active</i> <i>At or above threshold?</i> <i>Activate</i>
---	---


```
/* wArr[i] remains 0.f; optimizer will update it ne  
xt step */  
    grown++;  
}  
}  
}
```

WARNING

After `rigLUpdateMask`: `pruned += grown = 0`. Sparsity is preserved exactly. The newly grown weights start at 0.0 — the optimizer gives them their first nonzero value in the next `optimizer.step()`. Pruned weights are set to 0.0 immediately (not waiting for re-mask).

Chapter 12

End-to-End Integration Walkthrough

Wiring all pieces together for LibriSpeech speaker ID on STM32 Nucleo-F746ZG

12.1 — Complete Memory Layout

```
/* Model: Linear(256, 128) -> ReLU -> Linear(128, 10) */
/* Only layer 0 is sparse */

/* Layer 0 sparse config */
static uint8_t maskData0[32768/8];
static tensor_t maskTensor0;
static shape_t maskShape0;
static size_t maskDims0[2] = {128, 256};
static size_t maskOrder0[2] = {0, 1};
static quantization_t maskQuant0;
static rigLSparsityConfig_t sparsityCfg0;

/* Model-level RigL config */
static rigLModelConfig_t rigLCfg = {
    .updateFreq = 100,
    .numCycles = 10,
    .turnoverFrac = 0.20f,
    .globalStep = 0,
    .cyclesDone = 0,
};
```

4096 bytes = 4 KB

BOOL tensor struct

outFeatures x inFeatures

Update mask every 100 steps

10 prune+grow cycles then freeze

20% of active weights swapped

12.2 — Initialization

```
/* 1. Init mask tensor */
initBoolQuantization(&maskQuant0);
maskShape0.numberOfDimensions = 2;
maskShape0.dimensions = maskDims0;
maskShape0.orderOfDimensions = maskOrder0;
maskTensor0.data = maskData0;
maskTensor0.shape = &maskShape0;
maskTensor0.quantization = &maskQuant0;
maskTensor0.sparsity = NULL;

/* 2. Initialize to 90% sparse */
linearLayerInitSparse(&linearCfg0, 0.90f,
    &maskTensor0, maskData0,
    100, 10, &sparsityCfg0);
printf("Active weights after init: %zu\n",
    rigLCountActive(&maskTensor0));
```

quantization_t for BOOL

T=100, C=10

Should print ~3277

12.3 — Training Loop

```
for (size_t ep=0; ep<numEpochs; ep++) {
    for (size_t bi=0; bi<numBatches; bi++) {
        batch_t *b = loader.getBatch(&loader, bi);

        /* Forward + backward */
        calculateGradsSequential(model, 3,
                                &lossConfig, REDUCTION_MEAN,
                                b->samples[0]->item,
                                b->samples[0]->label);

        /* RigL mask update (if time) */
        rigLModelStep(model, 3, &rigLCfg);

        /* Optimizer step */
        sgdStepM(&optimizer);

        /* Re-mask: zero pruned positions */
        rigLReMaskModel(model, 3);

        /* Zero gradients */
        optimFns.zero(&optimizer);
        freeBatch(b);
    }
    /* Monitoring */
    printf("Epoch %zu | Active: %zu/%zu | Cycles: %zu/%zu\n",
        ep,
        rigLCountActive(&maskTensor0), 32768ul,
        rigLCfg.cyclesDone, rigLCfg.numCycles);
}
```

3 layers: Linear+ReLU+Linear

Uses grads from above

Updates ALL weights

MUST be after step

12.4 — Expected Training Dynamics

Phase	Steps	What happens	Active weights
Init	0	rigLInitMaskFromWeights prunes 90%	3277 / 32768
Warmup	1-99	Train with fixed sparse mask	3277
Cycle 1	100	Prune 655 + grow 655 ($K = 0.20 \cdot 3277$)	3277
Cycles 2-9	200-900	Repeat prune+grow each 100 steps	3277
Cycle 10	1000	Final mask update — cyclesDone=10	3277
Frozen	1001+	Train with fixed mask (rigLModelStep returns early)	3277 (fixed)

TIP

LibriSpeech 100h speaker ID benchmark with this config: dense baseline accuracy ~85%. RigL 90% sparse: ~83% (2% loss, 90% memory saved). This is the core claim of the Evci 2020 paper — sparse training matches dense accuracy.

Chapter Glossary

RigL Glossary — Every Term Defined

Complete reference for all RigL concepts, types, and functions

active weight	A weight where $\text{mask}[i]=1$. Participates in forward and backward pass.
inactive weight	A weight where $\text{mask}[i]=0$. Excluded from forward pass; value forced to 0 after each optimizer step.
BOOL tensor	Tensor with <code>BOOL</code> dtype: 1 bit per element, 8 elements packed per byte. Used for masks.
cycleCount	Number of completed prune+grow cycles. When $\geq \text{numCycles}$, mask is frozen.
cyclesDone	Alias for <code>cycleCount</code> in <code>rigLModelConfig_t</code> .
drift	Nonzero values in inactive positions ($\text{mask}=0$) after optimizer step. Cleared by <code>rigLReMask</code> .
findAbsKthSmallestActive	$O(k*N)$ $O(1)$ -space selection: finds k -th smallest $ w $ among active weights. Used for prune threshold.
findAbsKthLargestInactive	$O(k*N)$ $O(1)$ -space selection: finds k -th largest $ \text{grad} $ among inactive weights. Used for grow threshold.
getLayerMask	Returns the <code>BOOL</code> mask tensor for a layer, or <code>NULL</code> if dense.
globalStep	Total gradient steps since training started. Used to check: $\text{globalStep} \% \text{updateFreq} == 0$.
numCycles (C)	Total number of prune+grow cycles before the mask is frozen.
numToSwap (K)	$\text{floor}(\text{turnoverFrac} * \text{numActive})$. Weights pruned AND grown each cycle.
rigLApplyMask	Forces all inactive weights to 0.f. Called after <code>optimizer.step()</code> .
rigLCountActive	Counts number of set bits in mask tensor. $O(N/8)$ — iterates over bytes.
rigLInitMaskFromWeights	Prunes to <code>targetSparsity</code> by zeroing smallest $ w $ weights. Called once at init.
rigLInitSparsityConfig	Validates params and initializes <code>rigLSparsityConfig_t</code> . Exits on invalid input.
rigLLinearStep	Calls <code>rigLUpdateMask</code> for a Linear layer's weights.
rigLModelConfig_t	Model-level schedule: <code>T</code> , <code>C</code> , <code>Delta</code> , <code>globalStep</code> , <code>cyclesDone</code> .
rigLModelStep	Checks schedule; if time, calls per-layer <code>rigL*Step</code> . Increments <code>cyclesDone</code> .
rigLReMaskModel	Calls <code>rigLApplyMask</code> for all sparse layers after <code>optimizer.step()</code> .
rigLSparsityConfig_t	Per-layer config: <code>mask</code> , <code>targetSparsity</code> , <code>turnoverFrac</code> , <code>updateFreq</code> , <code>numCycles</code> , <code>stepCount</code> , <code>cycleCount</code> .
rigLUpdateMask	The combined prune+grow step: deactivates K smallest $ w $ active, activates K largest $ \text{grad} $ inactive.
targetSparsity	Fraction of weights that should be inactive. $0.90 = 90\%$ inactive, 10% active.

tensorBoolGet	Read one bit from a BOOL tensor: $(data[i/8] \gg (i\%8)) \& 1$.
tensorBoolSet	Write one bit to a BOOL tensor: set via $ =$, clear via $\& \sim$.
turnoverFrac (Delta)	Fraction of active weights swapped per cycle. $K = \text{floor}(\text{Delta} * \text{numActive})$.
updateFreq (T)	Steps between prune+grow updates. Mask updated when $\text{globalStep} \% T == 0$.
weightMask	Field added to <code>linearConfig_t</code> and <code>conv1dConfig_t</code> . NULL = dense layer.